



Missions CRUD

Mantenimiento de Software

Evolución del CRUD inicial · Robustez · Observabilidad · Seguridad

Node.js + Express

SQLite

Jest / Node:test

OpenAPI

Arquitectura del Sistema

Stack tecnológico y estructura de archivos

RUNTIME

Node.js ≥ 22.5

LOGGER

JSON estructurado

FRAMEWORK

Express 5.x

SEGURIDAD

API-Key Header

BASE DATOS

SQLite (nativo)

TESTS

Node:test nativo

```
└─ missions-crud/
  server.js
  package.json
  .env.example
  └─ test/
    mission-api.test.js
  └─ docs/
    openapi.json
  └─ postman/
    missions-crud.json
  CAMBIOS_APLICADOS.md
```

Endpoints CRUD

Las cuatro operaciones fundamentales de la API

POST

/missions

Crear misión con ID único MSN-XXXX



GET

/missions

Listar con filtros: status, agency, q...



GET

/missions/:id

Consultar misión por ID específico



PUT

/missions/:id

Actualizar campos bajo validación



DELETE

/missions/:id

Eliminación física de la misión



Logs Estructurados de Auditoría

JS

```
logger.info("AUDIT_CREATE", {  
  action: "CREATE",  
  method: req.method,  
  path: req.originalUrl,  
  ip: req.ip,  
  timestamp: new Date().toISOString()  
});
```

- ✓ Niveles INFO / WARN / ERROR
- ✓ Timestamp ISO 8601 en cada entrada
- ✓ Escribe en consola y logs/audit.log
- ✓ Sanitización de valores nulos con clean()
- ✓ Middleware de errores 500 centralizado

API-Key & Configuración Externa



Middleware requireApiKey

- 1 Request llega al servidor
- 2 Lee cabecera X-FIS-EPN-KEY
- 3 ¿Clave válida?
- ✓ → next() (continúa)
- ✗ → 401 Unauthorized

.env.example

```
PORT=4000

API_KEY_HEADER=X-FIS-EPN-KEY

FIS_EPN_API_KEY=fis-epn-2025

REQUIRE_API_KEY=true

MISSIONS_DB_PATH=./db/missions.sqlite

LOG_FILE_PATH=./logs/audit.log

SEND_EVENTS=true

CORS_ORIGINS=*
```

Suite de Pruebas & Documentación

✓ Validación Obligatoria

Rechaza si falta name, agency o type

✓ Entradas Maliciosas

<script>, DROP TABLE, SQL injection

✓ Seguridad API-Key

Sin X-FIS-EPN-KEY → 401 Unauthorized

✓ Ciclo CRUD Completo

Crear → Filtrar → Actualizar → Eliminar

 OpenAPI / Swagger

docs/openapi.json

 Colección Postman

postman/missions-crud.json

 CAMBIOS_APLICADOS.md

Deuda técnica documentada

Programación Defensiva

Validaciones en validateMission()

| | |
|-----------------------------------|---|
| <code>name / agency / type</code> | Obligatorios — error si vacíos |
| <code>name ≤ 80 chars</code> | textLimits controla longitudes |
| <code>status</code> | Solo: planned active completed failed aborted |
| <code>date</code> | Formato YYYY-MM-DD + Date.parse válido |
| <code><script></code> | hasBlockedContent → 10 patrones regex |
| <code>DROP TABLE</code> | SQL injection bloqueado |
| <code>UNION SELECT</code> | SQL injection bloqueado |

Error Handling

- ◆ `asyncRoute()` envuelve rutas async
- ◆ `app.use(error, req, res, next)` global
- ◆ try-catch en `writeStructuredLog()`
- ◆ 404 estandarizado para rutas no encontradas
- ◆ Consultas SQLite preparadas (no concatenación)

Guía de Ejecución

Pasos para correr, probar y demostrar el CRUD ahora mismo

1

Instalar dependencias

```
cd YO-SOY-CRUD/missions-crud
npm install
```

2

Configurar variables de entorno

```
cp .env.example .env
# Editar si necesitas cambiar puerto o API key
```

3

Correr pruebas unitarias

```
npm test
# Deben pasar los 4 tests 
```

4

Iniciar el servidor

```
npm start
# → http://localhost:4000
```

5

Probar endpoints (requiere header)

```
curl -H "X-FIS-EPN-KEY: fis-epn-2025"
http://localhost:4000/missions
```

6

Verificar logs de auditoría

```
cat logs/audit.log
# JSON con timestamps ISO 8601
```